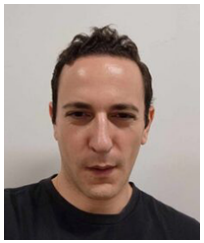
AI-Powered Vibe Coding with GitHub Copilot

AI Workflows for Biomedical Data Science



Anthony Christidis
Computational Scientist

Alex Pickering
Sr. Software Engineer

Core for Computational Biomedicine (CCB)
Department of Biomedical Informatics (DBMI)
Harvard Medical School (HMS)

April 14, 2026

HARVARD MEDICAL SCHOOL | BLAVATNIK INSTITUTE BIOMEDICAL INFORMATICS

Workshop Materials

Everything we do today is open-source.

- Slides
- Setup Guides
- Live Demo Scripts
- Hands-On Datasets & Starter Code

GitHub Repository:
`https://github.com/ccb-hms/ai-powered-coding`

Workshop Agenda

Hour 1: Concepts & Live Demo

- The AI Coding Landscape
- How AI Context Works
- *Find vs. Fix* Workflow
- Data Privacy
- **Live Coding Demonstration**

Hour 2: Hands-On Exercise

- TCGA Survival Analysis (R or Python)
- Guided AI Prompting
- Instructors available for help

What we *mean* by "Vibe Coding"

"Focusing on the overall outcome (the vibe) rather than writing syntax line-by-line."

— Coined by Andrej Karpathy (2025)

The AI "Killer App"

Software development is arguably GenAI's greatest success story. Anthropic's **Claude** currently dominates the coding space, forcing OpenAI into massive internal investments to catch up.

The Approach:

- **Natural Language:** Prompting AI to generate, debug, and iterate.
- **Rapid Prototyping:** Prioritizing speed and creativity to test ideas quickly.
- **Role Shift:** The developer becomes a **director**, not a typist.

What we *don't* mean by "Vibe Coding"

The Golden Rule of AI Coding

You cannot just "go with the vibe." You remain **strictly accountable** for understanding, reviewing, and validating all produced code.

Important Limitations:

- **No Blind Acceptance:** Never accept code you cannot read or explain.
- **Standards Still Apply:** Robust and secure best practices are mandatory prior to publication.
- **Prototyping $\neq$ Production:** Vibe coding is great for amateur prototyping, but deploying real software still demands engineering rigor.

The Vibe Coding Mindset: Offloading vs. Surrender

The Goal: Focus on scientific intent, not syntax.

Cognitive Offloading (GOOD)

- **AI is the typist.**
- Offload boilerplate, syntax, and formatting.
- Frees mental space for high-level scientific logic.

Cognitive Surrender (BAD)

- **AI is the scientist.**
- Blindly trusting AI-generated math or logic.
- Running code you cannot read or explain.

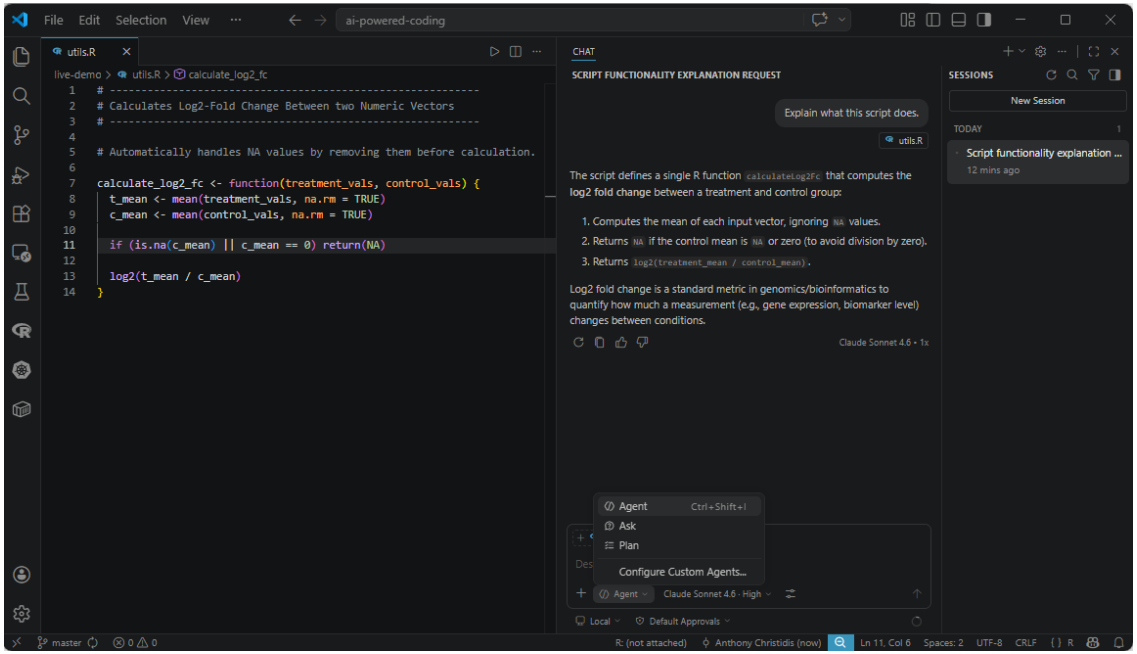
You remain the pilot. Copilot is just the navigator.

What is GitHub Copilot Today?

It is no longer just "autocomplete." It is an umbrella of integrated tools:

1. **Code Completion** (Ghost Text)
2. **Copilot Chat** (Sidebar & Inline)
3. **Copilot Agents** (Autonomous edits)

Plus: You can now choose your underlying AI model (Claude Sonnet 4.6, GPT-5.4, etc.).



The AI Coding Landscape

Copilot is the enterprise standard, but it is not alone.

Cursor

A standalone, AI-first editor. Currently the hottest competitor for power users.

GitHub Copilot

An extension added to your existing editor. The standard for enterprise & universities.

Continue (.dev)

Open-source extension. Allows you to run local, offline models (Ollama) for strict HIPAA privacy.

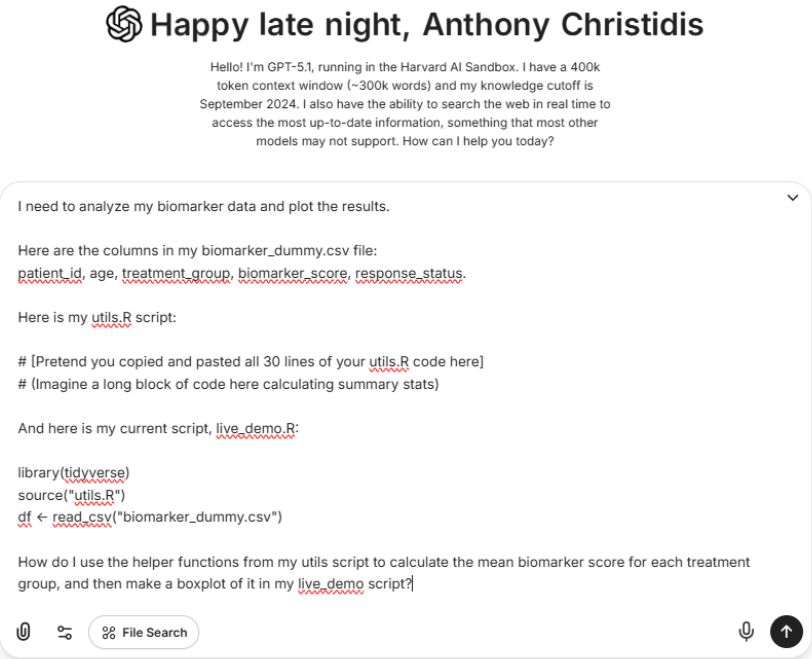
Browser Chat vs. IDE Integration

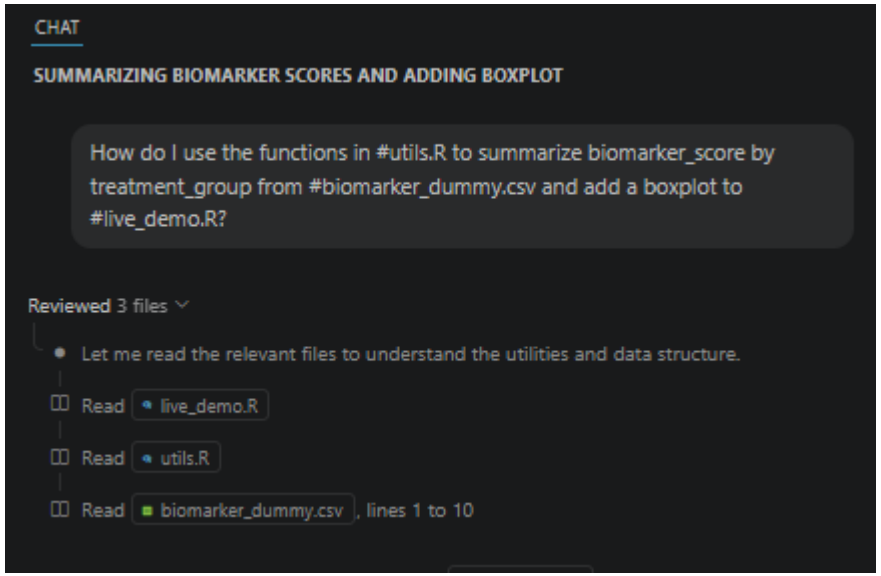
Browser (ChatGPT / Claude Web)

- Isolated from your work.
- Requires constant copy/pasting.
- Hallucinates variable names.

IDE Integration (Copilot)

- Context-aware.
- Sees your open tabs & folders.
- Writes code matching *your* environment.

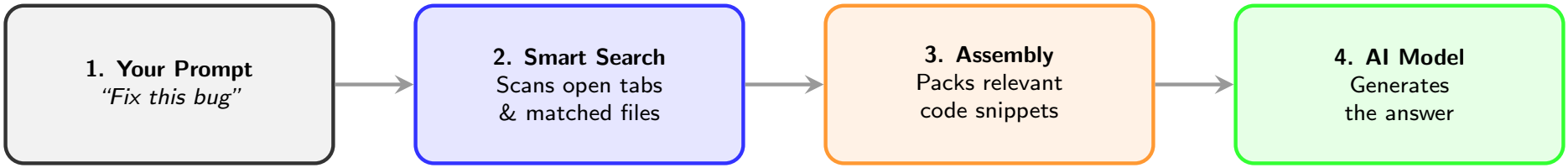




How Copilot Actually Works: The Context Window

Myth: *"Copilot memorizes my entire 10,000-line repository."*
Reality: AI has a strict memory limit (Context Window).

Retrieval-Augmented Generation (RAG) in Action:



If step 2 (Smart Search) guesses the wrong files, the AI will hallucinate.
This is why we manually direct its attention!

Best Practice: Pinning Context with

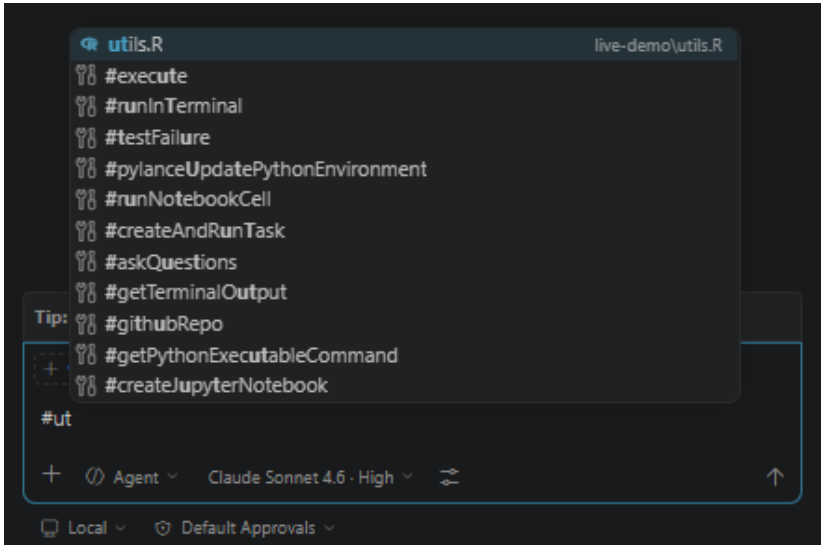
Copilot tries to automatically search your project, but it can guess the wrong files.

Take Manual Control

- Use the # symbol to explicitly "pin" exactly what the AI should read.

Most Useful Tags:

- `#filename` (e.g., `#utils.R`)
- `#selection` (Only reads highlighted code)
- `#terminalLastCommand` (Great for debugging errors!)



Interacting with AI: The Four Modes

Depending on your task, you can ask the AI to play different roles:

- **Ask:** Answers questions, explains code. (Read-only).
- **Edit:** Rewrites highlighted code in place (`Ctrl+I`). Shows a Red/Green Diff.
- **Plan:** Writes a step-by-step logic blueprint *before* generating code.
- **Agent:** Autonomously opens multiple files, writes code, and tests it in the terminal.

Choosing Your Environment

VS Code

- Industry standard.
- Most stable Copilot experience.

Positron

- Uses *Positron Assistant*.
- **Superpower:** Reads live session environment (dataframes!), not just text files.

RStudio

- Supports autocomplete only.

Git & Terminal Productivity

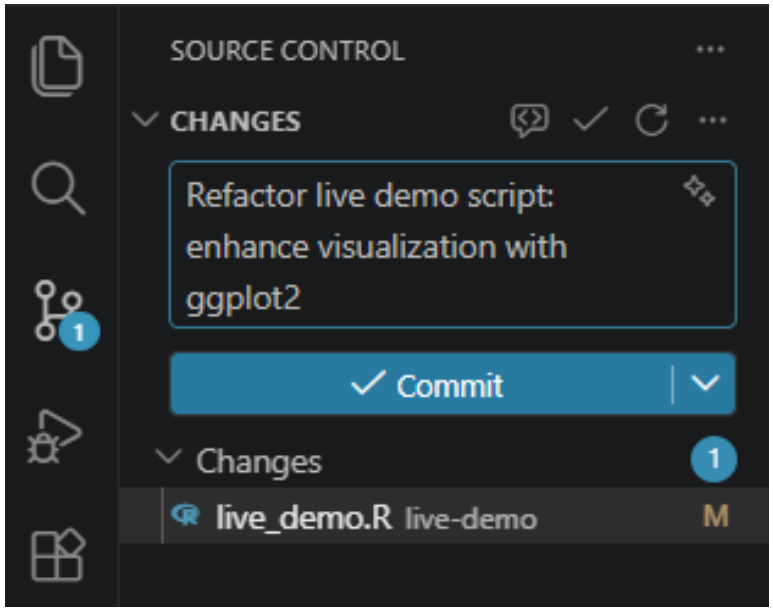
Copilot manages the environment, not just the script.

Terminal Assistance

- `@terminal How do I undo my last git commit?`

The "Magic Commit"

- Copilot reads your file diffs and auto-generates professional commit messages.



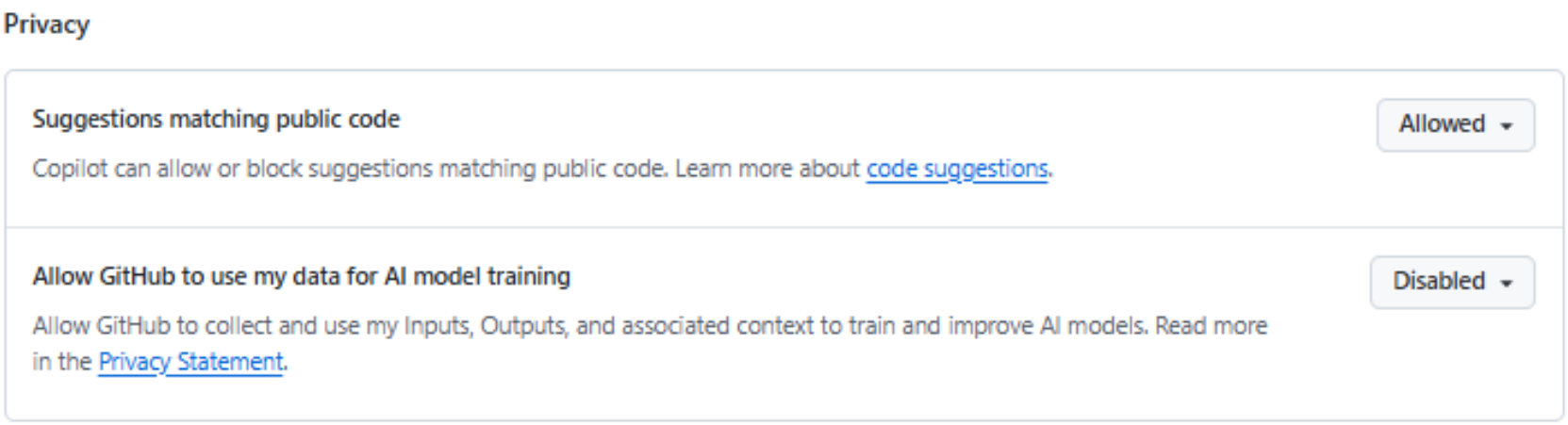
Data Privacy & Security at HMS

Critical Privacy Rule

NEVER use GitHub Copilot (or any cloud-based LLM) with Protected Health Information (PHI), Personally Identifiable Information (PII), or restricted clinical datasets.

Action Required:

- Go to GitHub Settings → Copilot.
- **UNCHECK:** *"Allow GitHub to use my data for AI model training."*



Beyond Copilot: AI *Inside* Your Code

AI can also act as a computational engine within your scripts.

LLMs as R/Python Functions:

- **API Endpoints:** Call models (Claude, GPT) directly from your script.
- **AI Data Manipulation:** Packages like `ellmer` (Posit) tidy data or run NLP on dataframes using plain English.

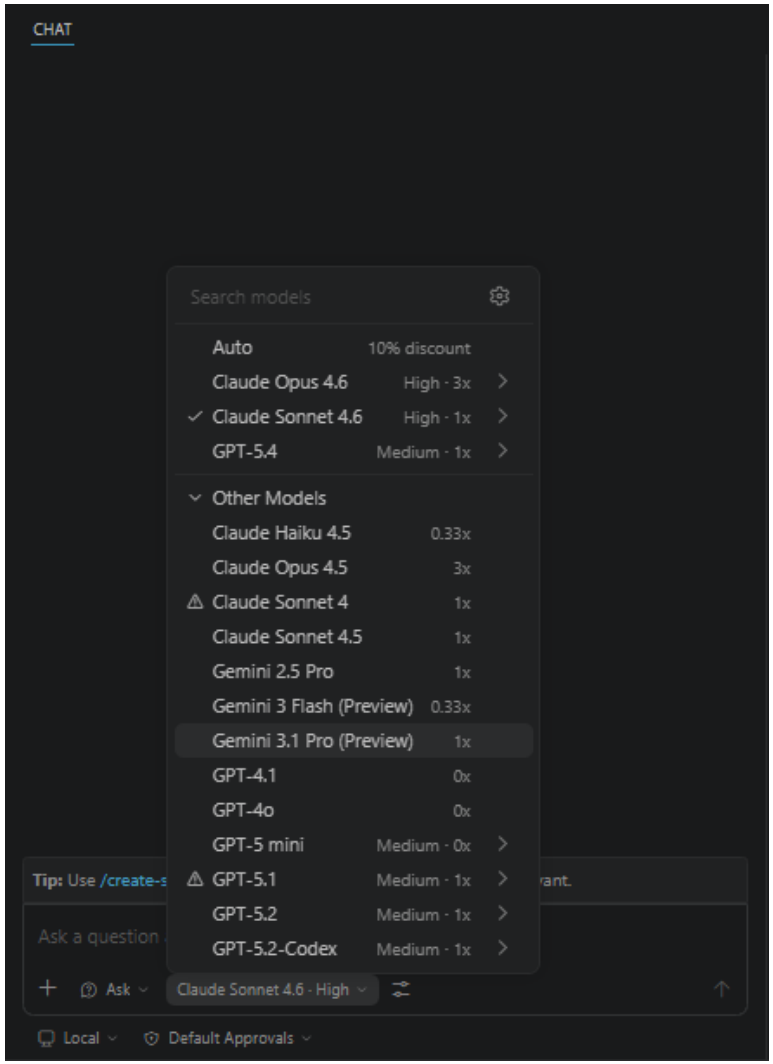
Resource:
Comprehensive List of R LLM Packages

Preparing for the Live Demo / Hands-On

Before we begin the Live Demo, ensure you have the best model selected.

- Open Copilot Chat.
- Click the Model Dropdown.
- Select **Claude 4.6 Sonnet**.

Note: RStudio users do not have this option.



Live Demonstration

Let's see it in action.